

Министерство образования Республики Беларусь

Учреждение образования  
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем  
и сетей Кафедра электронных  
вычислительных машин

Лабораторная  
работа №4 по курсу  
«Операционные системы и системное  
программирование» на тему  
«Задача производителя-потребители для процессов»

Выполнил:

студент группы 450501

М.В.Лебедь

Проверил:

старший преподаватель каф. ЭВМ

Л.П. Поденок

МИНСК 2026

## 1 УСЛОВИЕ ЛАБОРАТОРНОЙ РАБОТЫ

Основной процесс создает очередь сообщений, после чего ожидает и обрабатывает нажатия клавиш, порождая и завершая процессы двух типов – производители и потребители.

Очередь сообщений представляет собой классическую структуру – кольцевой буфер, содержащий указатели на сообщения, и пара указателей на голову и хвост. Помимо этого очередь содержит счетчик добавленных сообщений, счетчик извлеченных и количество свободного места в очереди.

Производители формируют сообщения и, если в очереди есть место, перемещают их туда.

Потребители, если в очереди есть сообщения, извлекают их оттуда, обрабатывают и освобождают память с ними связанную.

Для работы используются два семафора для заполнения и извлечения, а также мьютекс или одноместный семафор для монопольного доступа к очереди. Производители генерируют сообщения, используя системный генератор случайных чисел `rand(3)` или `rand_r(3)` для `size` и `data`. В качестве результата для `size` используется остаток от деления на 256. Реальный размер сообщения на единицу больше и лежит в интервале (1, 256).

Поле `data` имеет длину, кратную 4-м байтам. При формировании сообщения контрольные данные формируются только из байт сообщения длиной `size + 1`. Значение поля `hash` при вычислении контрольных данных принимается равным нулю.

В качестве семафоров используются семафоры System V.

После помещения значения в очередь перед освобождением мьютекса очереди производитель инкрементирует счетчик добавленных сообщений. Затем после освобождения мьютекса выводит строку на `stdout`, содержащую помимо всего новое значение этого счетчика.

Потребитель, получив доступ к очереди, извлекает сообщение и удаляет его из очереди.

Перед освобождением мьютекса очереди инкрементирует счетчик извлеченных сообщений. Затем после освобождения

мьютекса проверяет контрольные данные и выводит строку на `stdout`, содержащую помимо всего новое значение счетчика извлеченных сообщений.

При получении сигнала о завершении процесс должен завершить свой цикл и только после этого завершиться, не входя в новый.

Следует предусмотреть задержки, чтобы вывод можно было успеть прочитать в процессе работы программы. Следует предусмотреть защиту от тупиковых ситуаций из-за отсутствия производителей или потребителей.

Следует предусмотреть нажатие клавиши для просмотра состояния (размер очереди, сколько занято и сколько свободно, столько производителей и сколько потребителей).

## 2 ОПИСАНИЕ АЛГОРИТМОВ И РЕШЕНИЙ

Программа реализует классическую задачу «Производитель-Потребитель» с использованием многопроцессности. Взаимодействие осуществляется через механизмы System V IPC: разделяемую память для кольцевого буфера и набор семафоров для синхронизации.

Родительский процесс: Выполняет роль супервизора. Он создает ресурсы IPC, инициализирует управляющую структуру и в цикле ожидает команды пользователя. Родитель порождает дочерние процессы («р», «с»), ведет учет их PID в массиве и корректно завершает их работу («к», «q»), посылая сигнал SIGTERM.

Процесс-Производитель: В бесконечном цикле формирует сообщение: генерирует случайный тип, размер и данные, после чего рассчитывает контрольную сумму (хеш). Для записи в очередь производитель использует семафорную логику: ожидает свободный слот, захватывает мьютекс, копирует данные в общую память, инкрементирует счетчик добавленных сообщений и освобождает ресурсы.

Процесс-Потребитель: Ожидает появления данных в очереди. Захватив мьютекс, он извлекает сообщение, обновляет указатели кольцевого буфера и инкрементирует счетчик извлеченных сообщений. После освобождения мьютекса потребитель проверяет валидность хеша и выводит результат в stdout.

### 2.1 Основные системные вызовы и функции

В работе применяются следующие системные вызовы и функции:

1) `semget()` – создание набора из трех семафоров (MUTEX, FULL, EMPTY) с фиксированным ключом для синхронизации доступа к очереди.

2) `semop()` – выполнение атомарных операций над семафорами через функции-обертки `sem_wait()` (P-операция) и `sem_signal()` (V-операция).

3) `semctl()` – инициализация начальных значений семафоров (SETVAL) и удаление набора семафоров (IPC\_RMID) при завершении.

4) `shmget()` – создание сегмента разделяемой памяти для размещения структуры кольцевой очереди с сообщениями.

5) `shmat()` / `shmdt()` – присоединение сегмента разделяемой памяти к адресному пространству процесса и отсоединение от него.

6) `shmctl()` – удаление сегмента разделяемой памяти при завершении родительского процесса.

7) `fork()` – порождение дочерних процессов-производителей и потребителей.

8) `kill()` – отправка сигнала `SIGTERM` дочерним процессам для их корректного завершения.

9) `waitpid()` – ожидание завершения конкретного дочернего процесса, предотвращение появления процессов-зомби.

10) `signal()` – установка обработчиков сигналов `SIGINT` (для родителя) и `SIGTERM` (для дочерних процессов) для graceful shutdown.

11) `sleep()` – имитация реальной работы с задержками между операциями добавления и извлечения сообщений.

## **2.2 Реализация родительского процесса (parent)**

Родительский процесс является центральным управляющим компонентом программы. Его работа начинается с инициализации ресурсов межпроцессного взаимодействия System V. С помощью вызова «`shmget()`» выделяется сегмент разделяемой памяти, размер которого рассчитан на структуру управления очередью и буфер для сообщений. Далее с помощью «`semget()`» создается набор из трех семафоров для синхронизации доступа к очереди. После создания ресурсов родитель присоединяет разделяемую память через «`shmat()`» и инициализирует управляющую структуру, обнуляя указатели головы и хвоста очереди, счетчики добавленных и извлеченных сообщений, а также текущую загрузку. С помощью «`semctl()`» устанавливаются начальные значения семафоров: количество свободных мест равно размеру очереди, количество занятых мест равно нулю, а мьютекс изначально разблокирован.

Основная логика родительского процесса заключена в интерактивном цикле обработки команд пользователя. При вводе команды «р» или «с» родитель порождает дочерний

процесс-производитель или потребитель с помощью системного вызова «fork()», сохраняя его идентификатор в соответствующем массиве. Команда «kpl» позволяет завершить последнего созданного производителя, а «kcl» — последнего созданного потребителя. Для завершения процессов используется сигнал «SIGTERM», отправляемый через вызов «kill()», после чего родитель ожидает их завершения с помощью «waitpid()» для предотвращения появления процессов-зомби. Команда «s» выводит текущую статистику очереди, читаемую из разделяемой памяти: загрузку, количество добавленных и извлеченных сообщений, а также число активных производителей и потребителей. Команда «q» или нажатие «Ctrl+C» инициирует процедуру корректного завершения всей системы: родитель отправляет сигнал завершения всем дочерним процессам, дожидается их остановки, освобождает разделяемую память с помощью «shmctl()» и удаляет набор семафоров через «semctl()».

## **2.3 Формирование среды окружения дочернего процесса**

Родительский процесс работает в одном из двух режимов.

В режиме «\*» родитель чистит файл env, в котором перечислены имена переменных (например, SHELL, HOME, USER, PATH), формируется массив child\_env в формате «Имя = значение», используя getenv() для получения текущих значений, и передаёт этот массив в execve() как окружение для дочернего процесса child.

В режиме «+» родитель не формирует окружение сам, а лишь запускает child, передавая ему через аргументы командной строки символ «+». Дочерний процесс самостоятельно читает файл env, для каждого имени переменной вызывает getenv() и выводит полученные значения.

## **2.3 Реализация дочернего процесса**

Родительский процесс управляет жизненным циклом производителей и потребителей асинхронно, не блокируясь в ожидании их завершения, что позволяет пользователю

динамически изменять количество активных процессов в системе. Сами процессы взаимодействуют друг с другом опосредованно через общие ресурсы IPC, а координация их доступа к кольцевому буферу строится на основе трех семафоров. Семафор MUTEX обеспечивает атомарность операций записи и чтения, а также изменения счетчиков очереди. Счетный семафор EMPTY ограничивает производителей, блокируя их при переполнении очереди, а семафор FULL блокирует потребителей, когда очередь пуста. При завершении программы по команде q родительский процесс отправляет сигнал SIGTERM всем активным дочерним процессам, идентификаторы которых хранятся в массивах, после чего дожидается их завершения с помощью блокирующего вызова wait().

### **3 ФУНКЦИОНАЛЬНАЯ СТРУКТУРА ПРОЕКТА**

Программа организована в виде модульной системы, где родительский процесс управляет ресурсами и жизненным циклом системы, а дочерние процессы инкапсулируют логику производителей и потребителей.

1) `parent.c` – точка входа в программу, где реализован основной цикл управления производителями и потребителями, обработка пользовательских команд (добавление процессов, удаление последних, вывод статуса), а также инициализация и очистка ресурсов IPC.

2) `common.h` – заголовочный файл, содержащий прототипы всех функций, определения структур (`Message`, `SharedQueue`), макросы констант (`QUEUE_SIZE`, `MAX_PROC`, ключи для IPC), а также объявления глобальных переменных и обработчиков сигналов.

3) `ipc_utils.c` – файл с реализацией всех функций для работы с разделяемой памятью и семафорами (`create_shared_memory`, `attach_shared_memory`, `destroy_shared_memory`, `create_semaphores`, `init_semaphores`, `sem_wait`, `sem_signal`), а также функций для работы с очередью (`queue_push`, `queue_pop`), вычисления хеша (`calculate_DJB2`), генерации сообщений (`create_random_message`) и обработчиков сигналов.

4) `producer.c` – файл с логикой процесса-производителя, который генерирует случайные сообщения и помещает их в разделяемую очередь, синхронизируя доступ через семафоры.

5) `consumer.c` – файл с логикой процесса-потребителя, который извлекает сообщения из очереди, проверяет их целостность по хешу и выводит результат проверки.



## 4 ПОРЯДОК СБОРКИ И ИСПОЛЬЗОВАНИЕ

### 4.1 Сборка и запуск debug версии

```
tar -xgvf Лебедь_М.В._lab04.tar.gz
cd Лебедь_М.В./lab04/
make
./lab4
```

### 4.2 Использование программы

Сборку проекта осуществляется при помощи make. Можно запустить программу ./lab4. Программа выведет приглашение, затем программа ожидает ввода одного из символов «р», «с», «kpl», «kcl», «s», «q».

```
[Mixeld@archlinux LAB_4]$ make run
mkdir -p build
gcc -Wall -Wextra -g -Isrc -c src/parent.c -o build/parent.o
gcc -Wall -Wextra -g -Isrc -c src/ipc_utils.c -o
build/ipc_utils.o
gcc -Wall -Wextra -g -Isrc -c src/producer.c -o
build/producer.o
gcc -Wall -Wextra -g -Isrc -c src/consumer.c -o
build/consumer.o
gcc -Wall -Wextra -g -Isrc -o lab4 build/parent.o
build/ipc_utils.o build/producer.o bui
ld/consumer.o
./lab4
```

Commands:

```
p          - add producer
c          - add consumer
kpl        - kill last producer
kcl        - kill last consumer
s          - show status
q          - quit
Press Ctrl+C for graceful shutdown
> p
Producer 1 started (PID: 140529)
> [Producer 140529] Added message #1. In queue: 1
[Producer 140529] Added message #2. In queue: 2
[Producer 140529] Added message #3. In queue: 3
c
Consumer 1 started (PID: 140532)
> [Consumer 140532] Popped message #1. In queue: 2
[Consumer 140532] OK: Hash match
[Producer 140529] Added message #4. In queue: 3
```

```
[Consumer 140532] Popped message #2. In queue: 2
[Consumer 140532] OK: Hash match
[Producer 140529] Added message #5. In queue: 3
[Consumer 140532] Popped message #3. In queue: 2
[Consumer 140532] OK: Hash match
s
```

```
--- STATUS ---
```

```
Queue Load: 2/10
```

```
Added: 5, Extracted: 3
```

```
Producers: 1, Consumers: 1
```

```
-----
```

```
> [Producer 140529] Added message #6. In queue: 3
[Consumer 140532] Popped message #4. In queue: 2
[Consumer 140532] OK: Hash match
[Producer 140529] Added message #7. In queue: 3
[Producer 140529] Added message #8. In queue: 4
q
```

```
[Parent] Shutting down by user request...
```

```
[Parent] Stopping 1 producers...
```

```
[Parent] Stopping 1 consumers...
```

```
[Parent] Waiting for children to terminate...
```

```
[Producer 140529] Shutting down...
```

```
[Consumer 140532] Shutting down...
```

```
[Parent] Force killing producer 140529
```

```
[Parent] Force killing consumer 140532
```

```
[Parent] Cleaning up IPC resources...
```

```
[Parent] Done. Exiting.
```

## 5 ОПИСАНИЕ МЕТОДОВ ТЕСТИРОВАНИЯ И РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ ПРОГРАММЫ

### 5.1 Методы тестирования программы

Программа тестировалась вручную, а также через утилиту valgrind.

### 5.2 Результат тестирования

Результаты ручного тестирования:

```
[Mixeld@archlinux LAB_4]$ valgrind make run
==141045== Memcheck, a memory error detector
==141045== Copyright (C) 2002-2024, and GNU GPL'd, by Julian
Seward et al.
==141045== Using Valgrind-3.25.1 and LibVEX; rerun with -h for
copyright info
==141045== Command: make run
==141045==
mkdir -p build
gcc -Wall -Wextra -g -Isrc -c src/parent.c -o build/parent.o
gcc -Wall -Wextra -g -Isrc -c src/ipc_utils.c -o
build/ipc_utils.o
gcc -Wall -Wextra -g -Isrc -c src/producer.c -o
build/producer.o
gcc -Wall -Wextra -g -Isrc -c src/consumer.c -o
build/consumer.o
gcc -Wall -Wextra -g -Isrc -o lab4 build/parent.o
build/ipc_utils.o build/producer.o bui
ld/consumer.o
./lab4
```

Commands:

```
p      - add producer
c      - add consumer
kpl    - kill last producer
kcl    - kill last consumer
s      - show status
q      - quit
Press Ctrl+C for graceful shutdown
> p
Producer 1 started (PID: 141070)
> [Producer 141070] Added message #1. In queue: 1
[Producer 141070] Added message #2. In queue: 2
c
```

```
Consumer 1 started (PID: 141076)
> [Consumer 141076] Popped message #1. In queue: 1
[Consumer 141076] OK: Hash match
[Producer 141070] Added message #3. In queue: 2
[Consumer 141076] Popped message #2. In queue: 1
[Consumer 141076] OK: Hash match
[Producer 141070] Added message #4. In queue: 2
[Consumer 141076] Popped message #3. In queue: 1
[Consumer 141076] OK: Hash match
[Producer 141070] Added message #5. In queue: 2
[Consumer 141076] Popped message #4. In queue: 1
[Consumer 141076] OK: Hash match
[Producer 141070] Added message #6. In queue: 2
[Consumer 141076] Popped message #5. In queue: 1
[Consumer 141076] OK: Hash match
[Producer 141070] Added message #7. In queue: 2
kpl[Producer 141070] Added message #8. In queue: 3
[Consumer 141076] Popped message #6. In queue: 2
[Consumer 141076] OK: Hash match
```

```
Killing last producer (PID: 141070)
[Producer 141070] Shutting down...
Last producer terminated. Remaining producers: 0
> [Consumer 141076] Popped message #7. In queue: 1
[Consumer 141076] OK: Hash match
kcl
Killing last consumer (PID: 141076)
[Consumer 141076] Shutting down...
Last consumer terminated. Remaining consumers: 0
> q
```

```
[Parent] Shutting down by user request...
[Parent] Waiting for children to terminate...
[Parent] Cleaning up IPC resources...
[Parent] Done. Exiting.
```

```
==141045==
```

```
==141045== HEAP SUMMARY:
```

```
==141045==      in use at exit: 184,375 bytes in 1,470 blocks
```

```
==141045==    total heap usage: 2,809 allocs, 1,339 frees,
709,951 bytes allocated
```

```
==141045==
```

```
==141045== LEAK SUMMARY:
```

```
==141045==    definitely lost: 0 bytes in 0 blocks
```

```
==141045==    indirectly lost: 0 bytes in 0 blocks
```

```
==141045==    possibly lost: 0 bytes in 0 blocks
```

```
==141045==    still reachable: 184,375 bytes in 1,470 blocks
```

```
==141045==    suppressed: 0 bytes in 0 blocks
```

```
==141045== Rerun with --leak-check=full to see details of
leaked memory
```

```
==141045==
```

```
==141045== For lists of detected and suppressed errors, rerun
with: -s
```

==141045== ERROR SUMMARY: 0 errors from 0 contexts  
(suppressed: 0 from 0)